

Backend design and database architecture

A first draft of how the MVP backend could be put together: the components, how each major use case travels from input to output, and how the database would be laid out. This design is the main output so far. Written to be discussed and changed before real implementation starts.

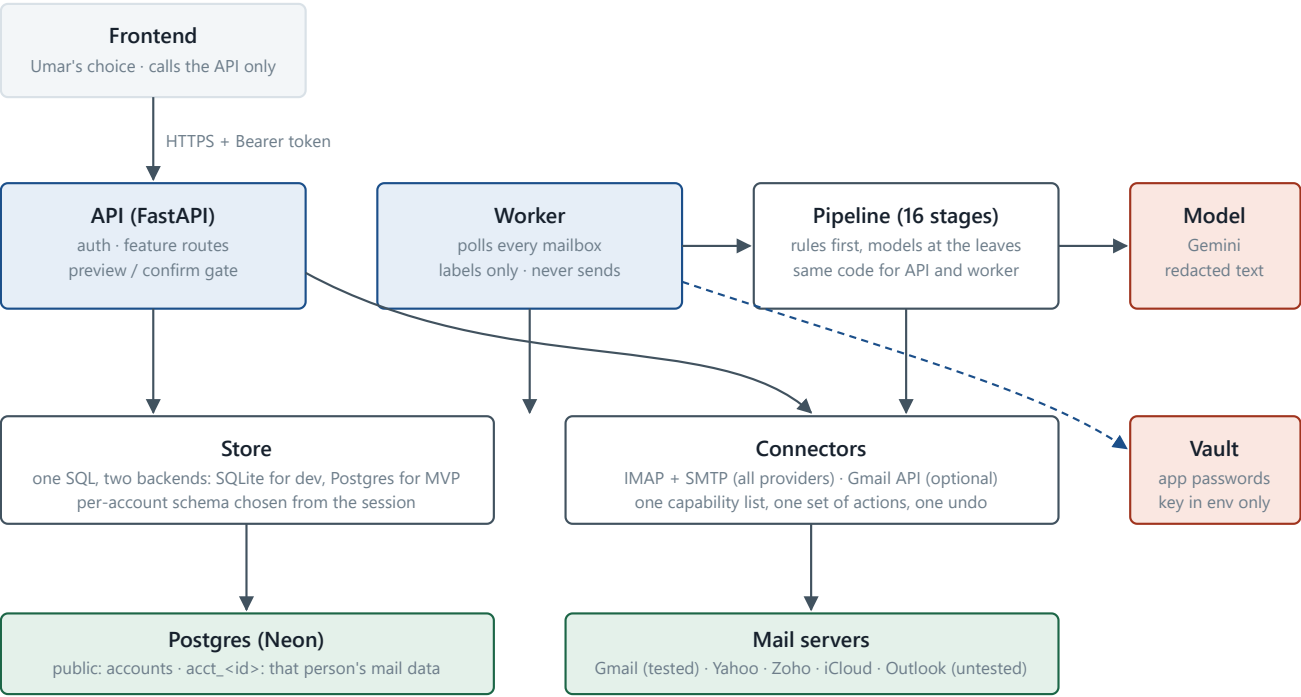
draft 0.2 · 23 September 2026 · Yahya Mateen · status: for review · implementation not started beyond exploratory checks

1 · What the backend has to do

The MVP is a web app where a person connects one or more mailboxes and Custodian reads, sorts and labels the mail, shows what needs attention, and lets the person act on it. The backend owns everything except the screens.

REQUIREMENT	WHERE IT COMES FROM
Accounts	Sign up, sign in, reset. A session reaches an account, never a mailbox on its own (UC-01, UC-02).
Mailboxes	Connect a mailbox by app password or provider sign-in; disconnect cleanly (UC-04 to UC-13). Target providers: Gmail (tested), Yahoo, Zoho and iCloud (standard IMAP, expected to work, not yet tried), Outlook (needs a Microsoft app registration and sign-in flow, not yet tried).
Always reading	New mail is noticed within seconds and passed through the pipeline without anyone at a keyboard.
Features	The 50 use cases: today's list, requests and promises, the pile, brands, unsubscribe, storage, spam rescue, reminders, rules, people, threads, search, digest, voice, calendar.
Safety	Nothing irreversible without the person seeing the exact wording and saying yes. Nothing is ever deleted. Credentials never in plain text. Model calls never see real names.
Separation	One customer's mail can never be read by a query meant for another customer.

2 · Components

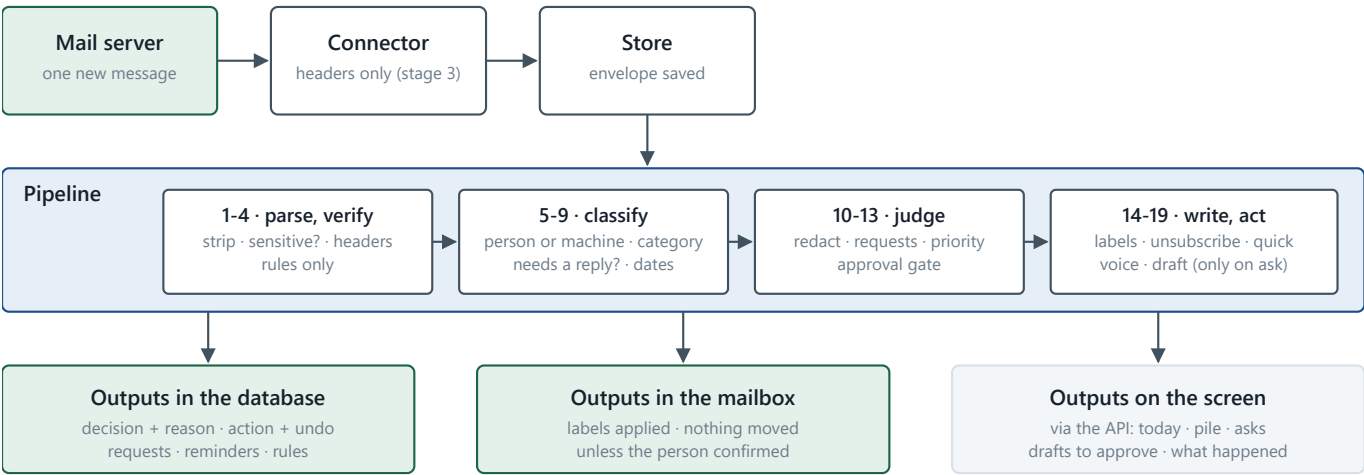


Solid lines are calls. Dashed: the worker reads each mailbox's app password from the vault; the API writes it there once at connect time. The API and the worker never call each other; they meet only in the database.

COMPONENT	RESPONSIBILITY	STATUS
API	Everything the frontend needs, as JSON. Checks the session, picks the account's store, calls the same functions the terminal scripts call. Holds the preview/confirm gate.	exploratory
Auth	Script password hashes, single-use confirm and reset tokens, session tokens, throttling on failed logins, one error message for wrong email or wrong password.	exploratory
Worker	One process, no browser. New mail first, a little backlog when idle, hourly housekeeping (requests scan, unsubscribe outcomes, digest). Cannot send, cannot clear.	exploratory
Pipeline	16 deterministic stages; models only where rules cannot decide. Unchanged from the POC. Redaction before every model call, restoration after.	POC code · to be reviewed
Store	One SQL dialect written once. SQLite for local work, Postgres for the MVP through a thin translation layer. Caches the handful of lookups that repeat per page.	exploratory
Connectors	IMAP/SMTP for every provider, Gmail API as an optional door. Same method names, same undo for each action, capability flags so callers never guess.	POC code · to be reviewed
Vault	App passwords encrypted with a key that exists only in the server's environment. Verified against the mail server before being stored.	exploratory

Outbound email	Confirmation and reset emails to the account holder. No service chosen yet; tokens are currently returned to the caller for testing.	not started
Chat	Sentence in, plain answer out, over the same feature functions. Design not started; only the typed-rules box exists.	not started
Deployment	API and worker hosted beside the database. Nothing is deployed; everything runs on one laptop.	not started

3 • Every message, end to end



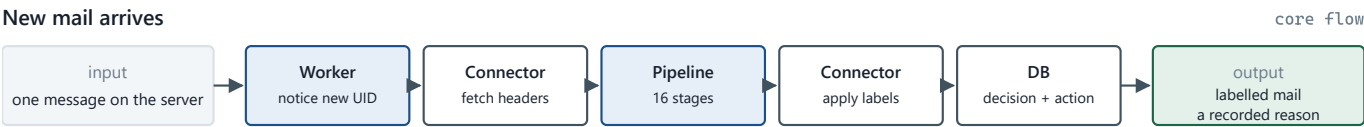
Every message takes this path. Earlier stages resolve most mail by rules alone; the model is only consulted where rules cannot decide, and only on redacted text.

4 • Major use cases, input to output

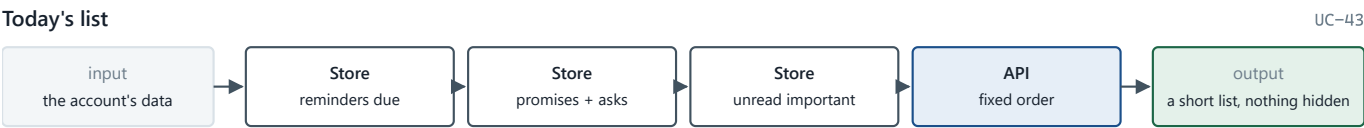
Each lane reads left to right: what comes in, which components touch it and in what order, what comes out. Red boxes are the gate; nothing to their right happens without the person's yes.



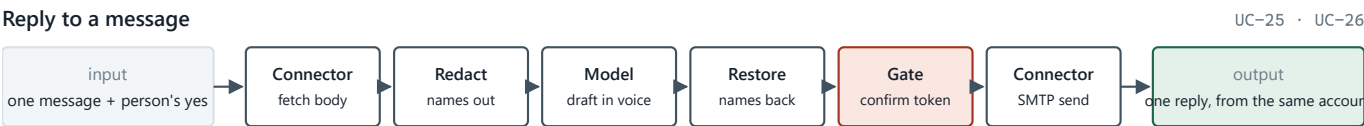
reversible: no approval needed, every step recorded with its undo



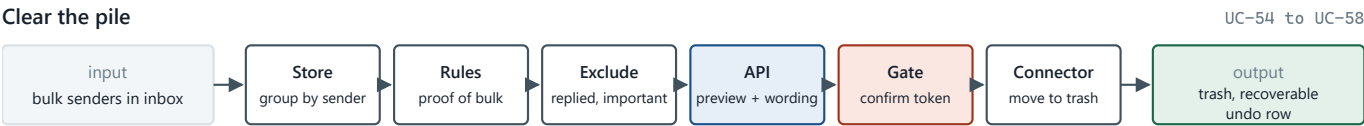
reversible: no approval needed, every step recorded with its undo



reversible: no approval needed, every step recorded with its undo



goes through the preview / confirm gate: refused without the person's yes



goes through the preview / confirm gate: refused without the person's yes

Unsubscribe

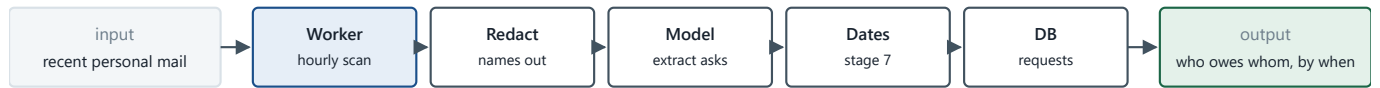
UC-27 · BR-66



goes through the preview / confirm gate: refused without the person's yes

Requests and promises

UC-33 · UC-45



reversible: no approval needed, every step recorded with its undo

Reminders and calendar

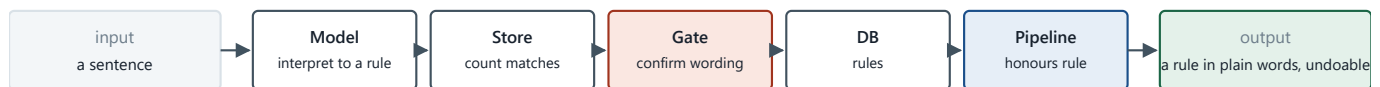
UC-30 · UC-46



reversible: no approval needed, every step recorded with its undo

Typed rules / important people

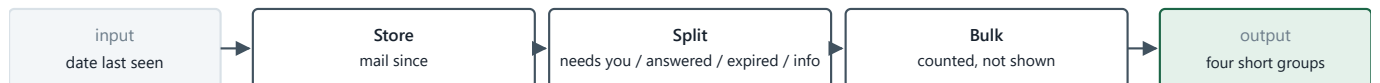
UC-38 · UC-44



goes through the preview / confirm gate: refused without the person's yes

Catch-up after time away

UC-42



reversible: no approval needed, every step recorded with its undo

Spam rescue

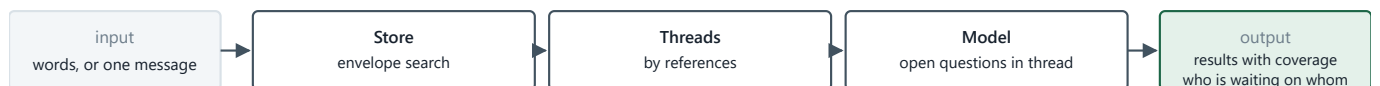
UC-31



reversible: no approval needed, every step recorded with its undo

Search and threads

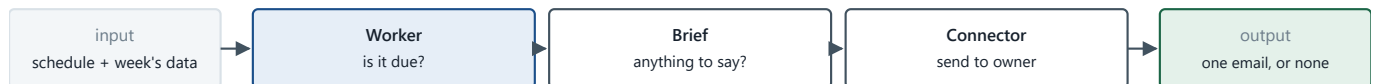
UC-28 · UC-40



reversible: no approval needed, every step recorded with its undo

Digest

UC-36



reversible: no approval needed, every step recorded with its undo

Disconnect a mailbox

UC-13

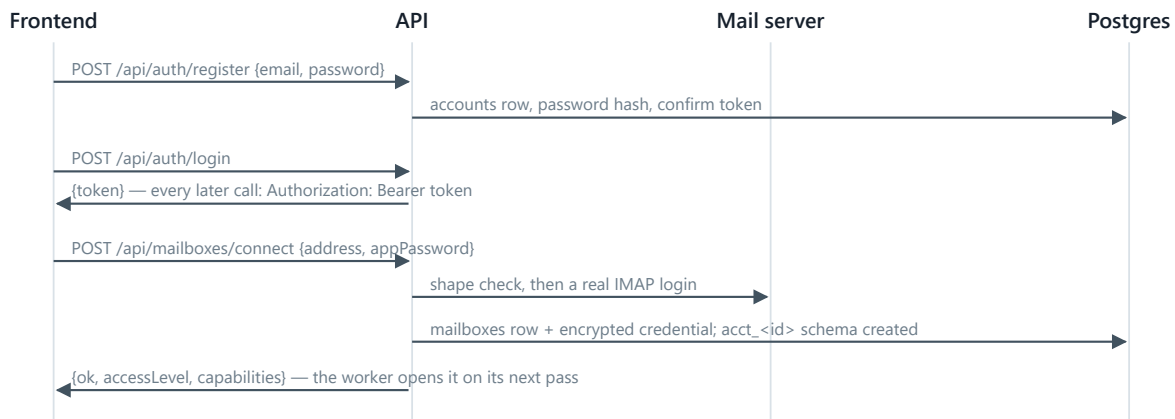


goes through the preview / confirm gate: refused without the person's yes

The account screens (UC-01 to UC-03, UC-47 to UC-49) are frontend flows over the auth routes and are not drawn here.

5 · How requests travel

5.1 · Sign in and connect a mailbox



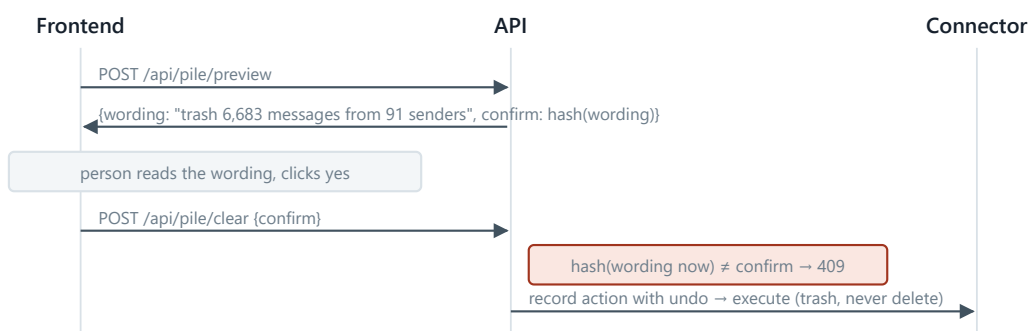
The password never goes to the log, never goes to the database in the clear, and is only stored after the mail server accepted it. If the server refuses, the person gets the provider's real reason, not a generic error.

5.2 · One pass of the worker

```
for each connected mailbox (own connection, own store)
  new = mailbox.new_since(bookmark)          ← highest UID (IMAP) or history id (Gmail)
  if new:
    for each message: save envelope → pipeline → labels applied → decision recorded
  else:
    scan 3 older messages                    ← backlog only when nothing new arrived
  once an hour:
    scan recent mail for requests and promises
    check whether unsubscribed senders actually stopped
    send the digest if it is due and there is something to say
sleep 20 s
```

The worker holds no approval, so it can only do reversible things. Everything it does is written to the actions table with how to undo it, before the call is made.

5.3 · An irreversible action



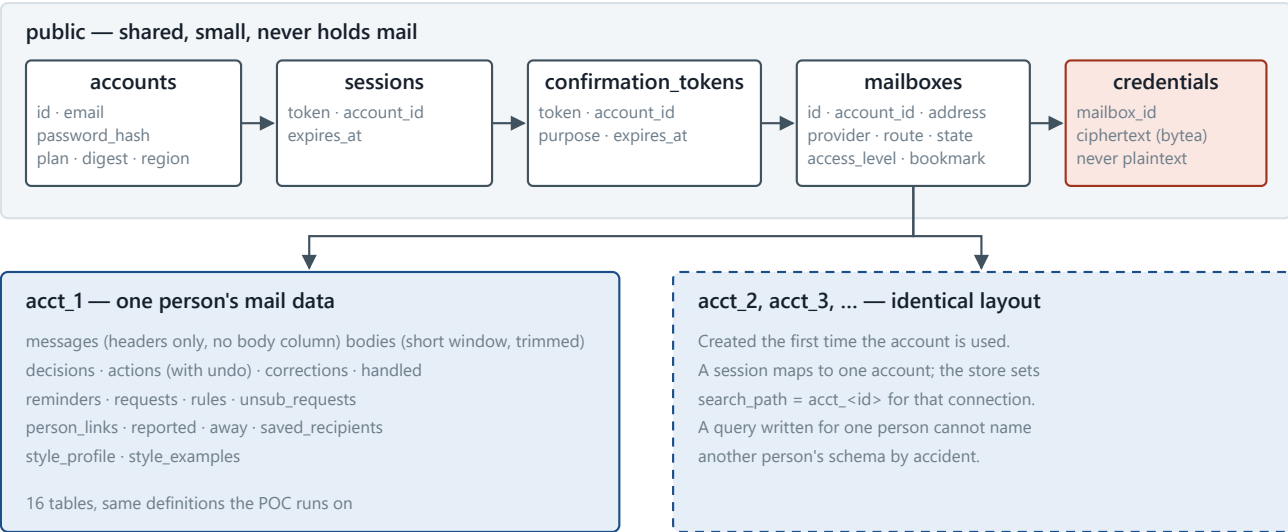
The token is bound to the wording. If the pile changed between preview and confirm, the wording changes, the token no longer fits, and the request is refused. This applies to clearing, unsubscribing, sending, batch

forwarding and creating rules. The same gate exists in the terminal version; the API reuses it rather than adding a second one.

6 • Database architecture

Two layers. A small shared layer that knows *who* the customers are, and one private schema per **account** that holds everything *about* that person's mail.

Account, not mailbox. An account is one customer (one sign-in). A customer may connect several mailboxes, say a Gmail and a work Outlook. All of them live in that customer's one schema; the `account` column on each message says which mailbox it came from. So "one schema per account" means one private area per customer, never one per mailbox. Ten thousand customers with three mailboxes each is ten thousand schemas, not thirty thousand.



Why one schema per account

OPTION	HOW IT WORKS	WHY NOT / WHY YES
account_id column	All customers in the same tables, every query filtered by account_id.	Common and fast to set up. One forgotten WHERE exposes another customer's mail. Rejected for now because the privacy promise is the product.
One schema per account proposed	Same tables, one namespace per account. The connection is pointed at the account's schema once, from the session.	Separation is structural, not a habit. Every existing script keeps its SQL unchanged. Fine to several thousand accounts; would need revisiting far beyond that.
One database per account	Full physical separation.	Strongest separation, but connection counts and migrations become a problem early. Too heavy for an MVP.

Rules the schema enforces

- **No message body column.** Bodies live in a separate table, kept for a short window and trimmed on start. Headers are enough for everything except drafting.
- **Credentials in their own table,** encrypted before they arrive. A dump of any other table never contains one.

- **Every action row carries its undo** and is written before the call is made. If the process dies in between, the record errs toward "it may have happened".
- **Nothing is ever deleted from a mailbox.** Trash is a move; the provider keeps it recoverable.
- **Message-ID is the key**, not the provider's id, so the same mail seen through IMAP and Gmail API is one row.

7 • Where things run

PIECE	TODAY	PROPOSED FOR THE MVP
Database	Neon (hosted Postgres), US East. Measured from Pakistan: the same query took 1 s one moment and up to 39 s the next. Trimming the query helped; distance did not go away.	Neon project in Singapore or Frankfurt. Same schema, re-import takes minutes.
API + worker	One laptop.	One small server in the same region as the database, so the chatty part of the traffic stays inside one datacentre. The link to the mail servers is the only long hop.
Local development	SQLite file, same SQL, no network.	Unchanged. The store picks the backend from the environment.
Secrets	Environment variables on the laptop.	Environment on the server: database URL, model key, vault key. None in the repo, none in the database.

8 · Resources: each model, at each number of users

Estimates from measurements on one laptop and one test mailbox, scaled by the assumptions in 8.2. Within a factor of two.

8.1 · Per model

MODEL	STAGE	RUNS ON	PARAMETERS	DISK	RAM LOADED	PER MESSAGE	LOAD ONCE
Classifier	8	our CPU	278 M	550 MB	1.7 GB	66 ms	49 s
Name spotter	10	our CPU	278 M	1.1 GB	1.0 GB	69 ms	12 s
Gemini Flash-Lite	7 · 18 · drafts · rules	Google	—	0	0	1 – 3 s	—
Voice profile	19	our CPU	—	<1 MB	<50 MB	<5 ms	—
Chat	parked	—	—	—	—	—	—

8.2 · Per user per day

ITEM	VALUE
Mail received	120
Settled by rules, no model	84 (70 %)
Through both local models	36
Hosted calls	26 (18 requests · 5 deadlines · 3 drafts)
Hosted tokens	42 k in · 4.5 k out
Local CPU time	5.4 s
Database growth	120 KB (1 KB per message, measured: 7,391 messages = 7.5 MB)
Mailbox polls	4,320 (every 20 s, 0.3 s each)

8.3 · Per number of users

USERS	SERVERS	VCPU TOTAL	RAM TOTAL	LOCAL MODEL CPU / DAY	DB SIZE (YEAR 1)	DB COMPUTE	HOSTED TOKENS / MONTH	HOSTED COST / MONTH	TOTAL / MONTH
10	1	4	8 GB	1 min	0.2 GB	free tier	13 M in · 1.4 M out	\$0 – 2	€15 – 25

100	1	4	8 GB	9 min	2 GB	0.25 CU	130 M in · 14 M out	\$20	€45 – 60
1,000	3	20	40 GB	1.5 h	20 GB	1 – 2 CU	1.3 B in · 140 M out	\$190	€300 – 400
10,000	24	190	380 GB	15 h	200 GB	4 – 8 CU	13 B in · 1.4 B out	\$1,900	€2,800 – 3,500

Server unit: 4 vCPU / 8 GB (€11) or 8 vCPU / 16 GB (€20), Hetzner CPX, September 2026. 1 CU = 1 vCPU + 4 GB on Neon. Hosted at Gemini 2.5 Flash-Lite \$0.10 / \$0.40 per million tokens. Worker servers assume polling; push would cut the 1,000 and 10,000 rows. At 10,000 the one-schema-per-account layout needs revisiting (160,000 tables).

8.4 · API or our own servers: which is cheaper, per model, per size

For each model, the monthly cost of calling it as a hosted API against the monthly cost of running it on servers we rent, at each number of users. Same assumptions as 8.2.

Classifier and name spotter (278 M parameters each)

USERS	AS API (DEDICATED CPU ENDPOINT, 2 MODELS)	ON OUR SERVERS	CHEAPER
10	\$48 (2 × \$0.033/h, billed 24/7)	€0 extra: 3 GB of the €11 server	ours
100	\$48	€0 extra	ours
1,000	\$48 – 96 (a second replica at peak)	€0 extra: 1.5 CPU-hours a day inside the worker servers	ours
10,000	\$150 – 300 (3 – 6 replicas)	€40: two 8 vCPU / 16 GB servers as a model service	ours

Small models cost almost nothing to run and a fixed fee to rent. They are cheaper on our servers at every size, and the name spotter must stay with us anyway, since its job is to find the private parts before anything leaves.

Reading and drafting (today Gemini Flash-Lite; self-host option: an open 7 – 8 B model on a rented GPU)

USERS	TOKENS / MONTH	AS API (FLASH-LITE \$0.10 IN / \$0.40 OUT PER M)	ON OUR SERVERS (L4 GPU \$0.80/H ≈ \$580 / MONTH EACH)	CHEAPER
10	13 M in · 1.4 M out	\$0 – 2 (free tier)	\$580 (1 GPU, mostly idle)	API
100	130 M in · 14 M out	\$20	\$580 (1 GPU)	API
1,000	1.3 B in · 140 M out	\$190	\$580 (1 GPU, ~50 % busy)	API
10,000	13 B in · 1.4 B out	\$1,900	\$3,500 (6 GPUs)	API

break-even around 20,000 users at these prices, and only if the open model matches Flash-Lite on our prompts, which is unproven. Batch mode halves the API price and pushes break-even further out.

GPU throughput assumed ~1,000 tokens/s mixed in and out for a 7 – 8 B model on one L4, ≈ 86 M tokens a day. Spot or reserved GPUs are 30 – 60 % cheaper and move the numbers, not the verdict.

Chat understanding (parked)

USERS	AS API (FLASH-LITE, 10 TURNS A DAY)	ON OUR SERVERS (OUR TWO SMALL MODELS, RETRAINED)	CHEAPER
10	\$0 (free tier)	€0 extra	tie
100	\$3	€0 extra	ours, barely
1,000	\$30	€0 extra	ours
10,000	\$300	€0 – 40	ours

Cost is not what decides this one. The small-model route needs a few hundred labelled example sentences first; the API route needs none. Privacy and dependency favour ours; time to build favours the API.

Everything together

USERS	ALL HOSTED APIS	ALL ON OUR SERVERS	MIXED (SMALL MODELS OURS, READING AND DRAFTING BY API) PROPOSED
10	€60	€560	€15
100	€85	€570	€50
1,000	€400	€850	€350
10,000	€2,900	€4,500	€2,900

Servers, database and the model costs above, rounded. "All hosted" rents the small models as endpoints too; "all ours" rents GPUs for reading and drafting. The mixed row is what section 9.2 proposes. Prices September 2026: Gemini 2.5 Flash-Lite \$0.10 / \$0.40 per M tokens; Hugging Face dedicated CPU \$0.033/h, L4 GPU \$0.80/h; Hetzner CPX32 €10.99, CPX42 €19.99; Neon \$0.106/CU-h, \$0.35/GB-month.

9 • Decisions

9.1 • Decided, technical

1. **One schema per account** for customer separation (section 6).
2. **Per-user mailbox connections in the API.** Each request opens the caller's own mailbox from the vault, cached per account. The single shared connection in the exploratory code is a prototype shortcut and will not survive.
3. **Push over polling** where the provider supports it (Gmail push, IMAP IDLE), polling only as the fallback. This is what keeps the 1,000 and 10,000 user rows in section 8 down.

4. **Chat model chosen by measurement.** A fixed 50-sentence test set first; the smallest model that passes wins. Parked until then.
5. **Local models stay local.** The classifier and name spotter run on our servers; the name spotter never leaves, since its job is to find the private parts.

9.2 • Proposed starting setup, cheapest that works

ITEM	PROPOSAL	COST	WHY
Region	Singapore for database and server	—	nearest to the first users; Frankfurt if they turn out to be in Europe
Servers	one 4 vCPU / 8 GB server running API, worker and both local models	€11 / month	the 10-user row in 8.3; enough to 100 users
Database	Neon free tier, moved to Launch when storage passes 0.5 GB	€0, then usage-based	about 25 users' first year fit in the free tier
Hosted model	Gemini Flash-Lite, free tier	€0, then ~\$0.19 / user / month	the free tier covers the first handful of users
Confirmation and reset emails	a free transactional tier (Brevo 300 / day or Resend 3,000 / month) from a no-reply address on our domain	€0	no server to run, sender reputation handled for us
Retention	message bodies kept 30 days, headers and decisions kept; user may shorten	—	what the POC already does; 30 days covers drafting and catch-up
Total to start	about €11 – 15 / month until roughly 25 users, then the 100-user row in 8.3		

10 • What exists right now, plainly

This design is drawn from the proof of concept and from exploratory backend work done to check it holds together, on one laptop against one test mailbox: the shared tables were applied to a hosted Postgres, a copy of the test mailbox was loaded, and the sign-in and a few of the routes were tried. That code is exploratory, not production quality, and is expected to be reshaped or replaced after this review. Nothing is deployed, nothing has been used by more than one person, and nothing has been pushed to a shared repository.

Appendix A • Use-case coverage, backend

Status of the backend logic behind each of the 50 use cases. "Built" means the logic exists and has been run against the test mailbox; screens are the frontend's. "Partly" means the pieces exist but a step is untested or missing. "Not built" is billing.

40 built	7 partly	3 not built (billing)	50 total
--------------------	--------------------	---------------------------------	--------------------

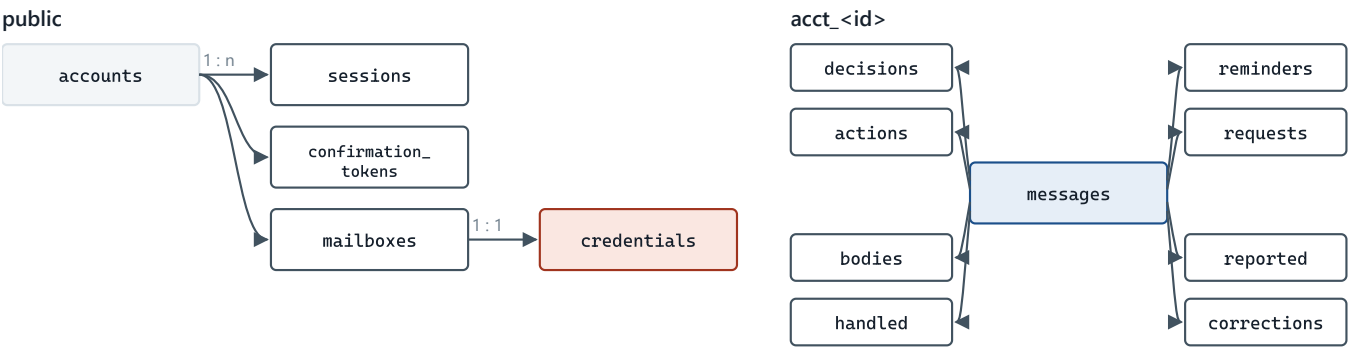
#	USE CASE	BACKEND	NOTE
UC-01	Register Account	built	email + password, confirm token; Google/Microsoft sign-in to the account not built
UC-02	Sign In	partly	email + password with sessions; provider sign-in not built
UC-03	Start a Subscription	not built	billing
UC-04	Gmail by App Password	built	the default door; tested
UC-05	Outlook by Microsoft Sign-In	partly	code exists; needs a Microsoft app registration; untested
UC-06	Yahoo by App Password	partly	server known; flow untested
UC-07	Zoho by App Password	partly	server known; flow untested
UC-08	iCloud by App-Specific Password	partly	server known; flow untested
UC-09	Any Other IMAP Mailbox	built	server from address, then MX; manual entry not built
UC-10	Establish Mailbox Connection	built	verify, folders, access level, capabilities
UC-11	Gmail by Google Sign-In	built	the API door
UC-12	Upgrade to Google Sign-In	built	
UC-13	Disconnect a Mailbox	built	credential and index erased; what is lost listed
UC-14	Scan Mailbox History	built	envelopes in bulk, resumable
UC-15	Review and Clear the Pile	built	gated
UC-16	Clear a Brand's Mail	built	gated
UC-17	Unsubscribe from a Sender	built	one-click only; outcome watched 14 days
UC-18	Reclaim Storage Space	built	
UC-19	Prioritise Newly Arrived Mail	built	the pipeline

UC-20	Recover Important Mail from Spam	built	never automatic
UC-21	Inspect a Suspicious Message Safely	built	
UC-22	Track Unanswered Outgoing Mail	built	Gmail API door only
UC-23	Record Commitments from Structured Mail	built	
UC-24	Warn About an Approaching Deadline	built	
UC-25	Generate a Reply Draft	built	redact, draft in voice, gated send
UC-26	Send a Quick Answer	built	three fixed answers, gated
UC-27	View the Unified Inbox	built	
UC-28	Search Across All Mailboxes	built	sender, subject, date; not message text
UC-29	Correct a Decision	built	
UC-30	Undo an Action	built	both doors
UC-31	Receive the Weekly Digest	built	
UC-32	Export or Erase My Data	built	
UC-33	Identify a Request and Its Deadline	built	
UC-34	Connect a Calendar	partly	Google only; live sign-in not exercised
UC-35	Forward a Batch to Someone	built	drafts only, recipient from the user
UC-36	Resurface an Unread Important Message	built	
UC-37	Set a Follow-Up Reminder	built	
UC-38	Create a Rule in Plain English	built	interpretation confirmed, then no AI
UC-39	Catch Up After Time Away	built	
UC-40	See Where a Conversation Stands	built	
UC-41	See Everything About One Person	built	
UC-42	Tune My Writing Voice	built	

UC-43	Work Through Today	built	
UC-44	Mark a Person as Important	built	
UC-45	Track What You Promised	built	
UC-46	The First Cleanup	built	
UC-47	Where My Data Is Stored	partly	region recorded and shown; move on request not built
UC-48	Change Plan	not built	billing
UC-49	Cancel Subscription	not built	billing
UC-50	Identify the Mail Provider	built	known endings, then MX records

Appendix B • Database schema

Every table and column. The shared layer is Postgres DDL in `db/schema.sql` . The per-account tables are the same definitions the proof of concept runs on SQLite, created in Postgres by the store the first time an account is used.



joined by message_id (the RFC Message-ID), so the same mail seen through IMAP and the Gmail API is one row

also in the account schema, not keyed by message:
rules · unsub_requests · person_links · away · saved_recipients · style_profile ·

B.1 • Shared layer, schema public

accounts • one customer

id	bigserial, key
email	text, unique
password_hash	text, scrypt; null for provider sign-in
confirmed_at	timestamptz
data_region	text
plan	text: trial · ...
digest	text: daily · weekly · monthly · never
digest_last	timestamptz
created_at	timestamptz

sessions • a signed-in browser

token	text, key
account_id	→ accounts.id
created_at	timestamptz
expires_at	timestamptz

confirmation_tokens • single-use links

token	text, key
account_id	→ accounts.id
purpose	text: confirm · reset
expires_at	timestampz

mailboxes · one connected address

id	bigserial, key
account_id	→ accounts.id
address	text, unique across all accounts
provider	text: gmail · outlook · yahoo · zoho · icloud · other
route	text: app_password · google_signin · microsoft_signin
host	text
access_level	text: full · read only
state	text: connected · paused · disconnected
state_reason	text
folders	jsonb: sent, drafts, spam, trash, archive
capabilities	jsonb
connected_at	timestampz
bookmark	text: history id or highest UID

credentials · the encrypted app password

mailbox_id	→ mailboxes.id, key
kind	text: app_password · oauth_refresh
ciphertext	bytea, encrypted before it arrives
updated_at	timestampz

schema_versions · migrations applied

version	integer, key
applied_at	timestampz
note	text

B.2 · Per-account layer, schema acct_<id>

messages · one row per message, headers only, no body

provider_id	text, key: the provider's id
message_id	text: the RFC Message-ID, the real identity
thread_id	text
account	text: which mailbox it came from
sender	text
sender_name	text
sender_domain	text
recipients	text
subject	text
date	text
date_iso	text, sortable
size	integer, bytes
unread	0/1
in_inbox	0/1
categories	text
bulk	0/1: Precedence bulk
unsubscribe	text: List-Unsubscribe header
one_click	0/1
list_id	text
auto_sub	text: Auto-Submitted
in_reply_to	text
refs	text: References
auth_results	text: SPF/DKIM/DMARC verdict

bodies · message text for a short window, then trimmed

message_id	text, key
provider_id	text
account	text
text	text

chars	integer
fetches_at	text
decisions · what each stage decided and why	
message_id	text
stage	text
decision	text
reason	text
score	real
at	text
key	(message_id, stage)
actions · everything done to the mailbox, with its undo	
id	serial, key
message_id	text
provider_id	text
action	text: label · archive · trash · send · ...
detail	text
before	text: what to restore
undone	0/1
at	text
corrections · the person disagreed	
id	serial, key
scope	text: sender · domain · ...
target	text
was	text
should_be	text
source	text
at	text
handled · replied / drafted, once	

message_id	text
action	text
sent_id	text
at	text
key	(message_id, action)

reminders · follow-ups and deadlines

message_id	text
provider_id	text
kind	text: remind · deadline
due	text
subject	text
why	text
done	0/1
key	(message_id, kind)

requests · asks of the person and promises by them

id	serial, key
message_id	text
provider_id	text
account	text
direction	text: ask · promise
what	text
who	text
due	text
evidence	text
confidence	real
status	text: open · done · dismissed
at	text

rules · typed rules and important-person rules

id	serial, key
sentence	text: as typed
match_kind	text: person · domain · subject
match_value	text
action	text
action_arg	text
plain	text: shown back
group_id	text
at	text

unsub_requests · unsubscribes sent and their outcome

sender	text, key
requested_at	text
outcome	text: watching · stopped · ignoring
checked_at	text

person_links · addresses that are the same person

address	text, key
person	text
evidence	text
by_owner	0/1
at	text

reported · surfaced once, not again

message_id	text
kind	text
at	text
key	(message_id, kind)

away · time-away periods for catch-up

id	serial, key
since	text

until	text
stated	0/1
done	0/1

saved_recipients · addresses the person typed for forwarding

address	text, key
label	text
at	text

style_profile · how the person writes

key	text, key
value	text

style_examples · sent messages used for voice

id	serial, key
text	text
subject	text
embedding	bytea; pgvector in production

Text columns hold ISO dates in the account layer because the same definitions run on SQLite for local development. Sizes measured: about 1 KB per message across all tables.

Custodian · backend design draft 0.2 · 23 September 2026 · related: docs/API.md (route contract), poc/db/schema.sql (shared tables), docs/use-cases-in-brief.md (use-case status), Doc1 Components (pipeline and connectors)